

RELATÓRIO TÉCNICO DE PROCESSAMENTO DE DADOS EM LARGA ESCALA COM APACHE SPARK

PIPELINE DISTRIBUÍDO, OTIMIZAÇÃO DE PERFORMANCE E ARMAZENAMENTO EM DELTA LAKE

Curso: Pós-Graduação em Data Science e Analytics

Unidade Curricular: Banco de Dados e Big Data para Data Science

Unidades de Aprendizagem: UA 03 e UA 04

Equipe de Desenvolvimento:

- **Diogo Galvão Carvalho**
 - **Felipe Artur Macedo Lima**
 - **Luan Cavalcante Dias Rodrigues**

Repositório do Projeto: https://github.com/FelipeArtur/bigdata_datascience

Data: Setembro de 2026

1. INTRODUÇÃO

1.1 Objetivo Geral

O objetivo deste trabalho é projetar, implementar, otimizar e documentar a jornada integral de um conjunto de dados complexo de comércio eletrônico no segmento gamer — a Loja Virtual de *League of Legends* — transitando desde sua concepção relacional e migração para NoSQL (**MongoDB Atlas**) até o processamento analítico em larga escala (*Big Data Analytics*) em um ambiente distribuído de alto desempenho utilizando o **Apache Spark** hospedado na plataforma **Databricks Community Edition**, com armazenamento final em formato **Delta Lake**.

1.2 Contextualização Técnica: Da Transação ao Processamento Distribuído

Na primeira etapa do projeto (UA 01 e UA 02), consolidamos a transição do modelo relacional (PostgreSQL) para o banco de dados orientado a documentos (MongoDB Atlas). Esse movimento resolveu a latência de leituras transacionais da aplicação cliente através de desnormalização controlada e *embedding* de compras dentro do perfil do jogador. Contudo, em organizações modernas de dados, a camada operacional (OLTP) é apenas a primeira etapa do ciclo de vida da informação.

Quando analistas de negócio e cientistas de dados precisam extrair inteligência a partir de bilhões de eventos de compras cruzados com variáveis demográficas, métricas de jogo e sazonalidade temporal, a execução de queries analíticas pesadas (OLAP) diretamente no cluster transacional NoSQL torna-se inviável. Essa prática compromete o isolamento de recursos (*noisy neighbor effect*), satura conexões de rede e degrada os tempos de resposta da loja para os usuários finais.

Nesse cenário, emerge a imperativa necessidade de um motor de computação distribuída em memória capaz de federar múltiplas fontes de dados heterogêneas. O **Apache Spark** destaca-se como o padrão da indústria para essa tarefa, fornecendo:

1. **Computação em Memória (In-Memory Processing):** Ao substituir o modelo de persistência intermediária em disco do Hadoop MapReduce por Grafos Acíclicos Dirigidos (DAGs) que mantêm partições de dados na memória RAM dos nós executores, o Spark atinge acelerações de até duas ordens de grandeza em pipelines analíticos;

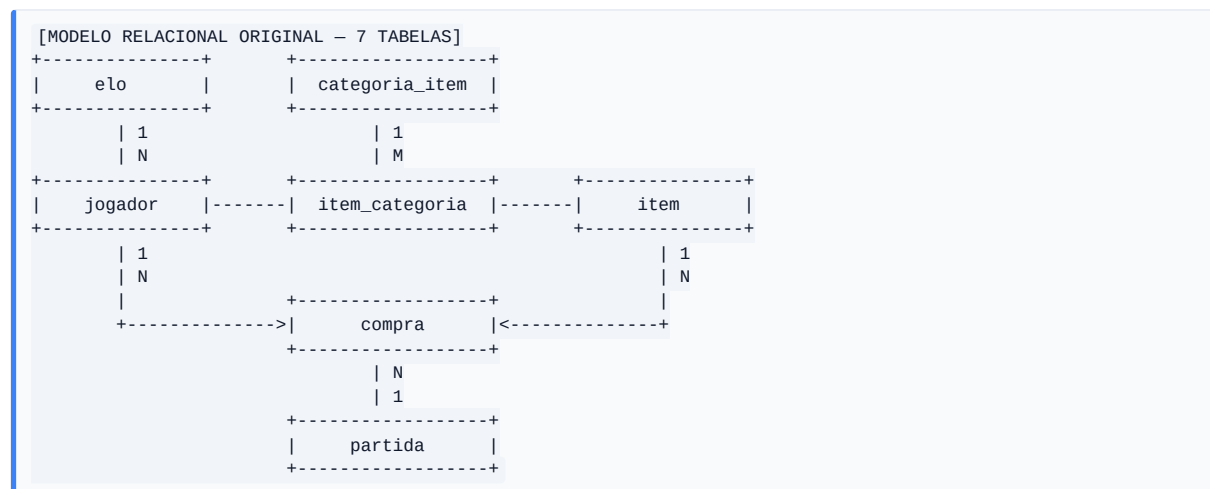
2. **Otimizador Catalyst e Motor Tungsten:** Um mecanismo avançado de compilação que reescreve grafos lógicos em planos de execução físicos altamente eficientes, aplicando poda de projeções (*projection pruning*), empurrão de predicados (*predicate pushdown*) e geração dinâmica de bytecode Java em tempo de execução;

3. **Arquitetura de Lakehouse com Delta Lake:** A convergência entre a flexibilidade de custo e armazenamento de um Data Lake com as garantias de governança, atomicidade transacional ACID e indexação inteligente típicas de Data Warehouses corporativos.

2. DESCRIÇÃO DO CENÁRIO E MODELO DE DADOS (RETOMADA DAS UAS 1 E 2)

2.1 Cenário Relacional de Origem (Retomada da UA 1)

O domínio de negócio modela as transações de compra de itens virtuais por jogadores durante partidas competitivas (5v5) de *League of Legends*. Na concepção relacional inicial, o sistema era composto por 7 tabelas normalizadas em 3ª Forma Normal:



- **elo` (10 registros):** Domínio de ranques do jogo (Ferro, Bronze, Prata, ..., Desafiante).
- **categoria` (32 registros):** Classificações de itens (Boots, ManaRegen, Damage, AbilityHaste, etc.).
 - **item` (254 registros):** Catálogo de itens reais do jogo, preços em ouro e descrições.
 - **item_categoria` (834 registros):** Tabela associativa que resolve o relacionamento N:N entre itens e categorias.
 - **jogador` (25 registros):** Competidores de diversas regiões (KR, BR, EUW).
 - **partida` (30 registros):** Confrontos com registro temporal, duração e resultado (Vitória/Derrota).
 - **compra` (1.043 registros):** Tabela fato que registra cada aquisição in-game, conectando jogador, item, partida, minuto, quantidade e valor total em ouro.

2.2 Modelo NoSQL Idealizado (Retomada da UA 2)

Na migração para o MongoDB Atlas, o modelo foi otimizado para o padrão de leitura da loja através de 3 coleções:

1. **Coleção jogadores (Embedding):** O elo e a lista completa de compras foram incorporados diretamente no documento do jogador. Cada compra contém o snapshot imutável do item adquirido (nome, categorias, preço unitário, quantidade e total), eliminando completamente a necessidade de `JOIN`s no acesso ao perfil do usuário.

2. **Coleção itens (Referencing):** O catálogo oficial de itens permaneceu como uma coleção referenciada autônoma, embutindo um array de tags de categoria `["Boots", "Speed"]`, o que extinguiu a tabela associativa relacional `item_categoria`.

3. **Coleção partidas (Referencing):** Coleção autônoma referenciada pelo atributo `id_partida` dentro de cada transação de compra.

```
// Exemplo de Documento BSON consolidado na coleção 'jogadores'
{
  "_id": 1,
  "id_jogador": 1,
  "nick": "Faker",
  "regiao": "KR",
  "elo": "Desafiante",
  "total_gasto_ouro": 80525,
  "compras": [
    {
      "id_compra": 72,
      "id_item": 3508,
      "item": "Colhedor de Essência",
      "categorias": ["Damage", "CriticalStrike", "ManaRegen", "AbilityHaste"],
      "preco_unitario": 3050,

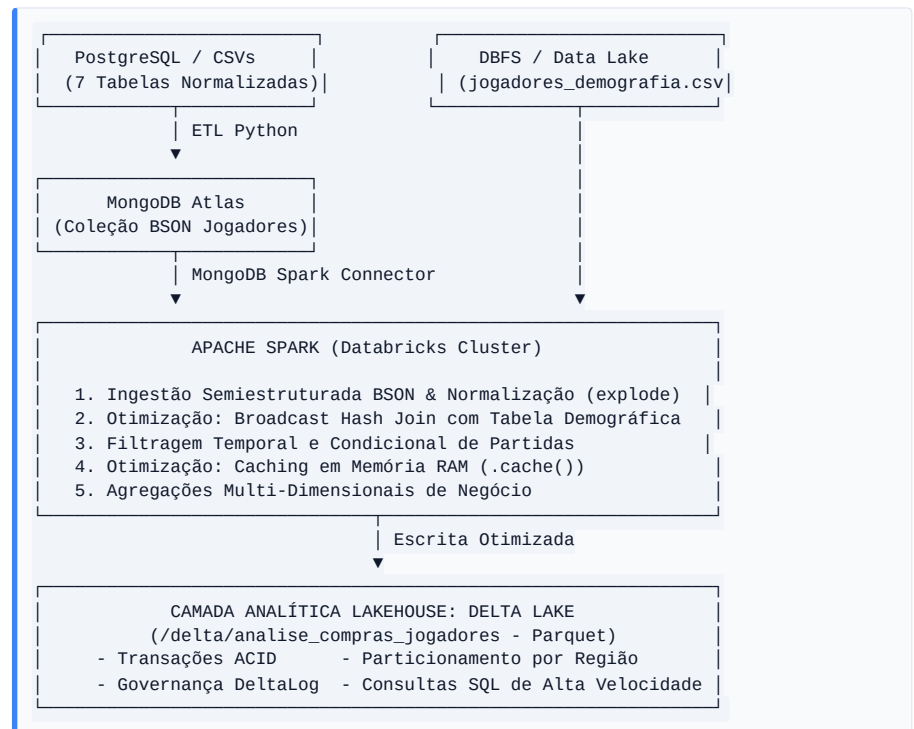
```

```
    "quantidade": 1,  
    "total_ouro": 3050,  
    "data_compra": "2025-08-21",  
    "minuto_compra": 18,  
    "id_partida": 3  
  }  
]  
}
```

3. ARQUITETURA DO PIPELINE DE PROCESSAMENTO DISTRIBUÍDO

3.1 Fluxo de Dados Fim a Fim

O pipeline analítico orquestra o ciclo completo de ingestão, enriquecimento, computação analítica e persistência Lakehouse conforme ilustrado a seguir:



3.2 Justificativa da Escolha das Tecnologias

- ****Databricks Community Edition:**** Plataforma unificada de dados gerenciada em nuvem que elimina a complexidade operacional de provisionamento manual de clusters Hadoop/YARN. Fornece instâncias Spark pré-otimizadas com runtime de alto desempenho, suporte nativo a notebooks interativos colaborativos e interface gráfica intuitiva de monitoramento do Spark UI (visualização de DAGs, tarefas e métricas de shuffle).
- ****MongoDB Spark Connector:**** Biblioteca oficial mantida pela MongoDB e Databricks que mapeia coleções BSON diretamente para DataFrames do Spark, convertendo esquemas aninhados complexos e permitindo empurrão de filtros (*predicate pushdown*) diretamente para o motor do banco NoSQL.
- ****Delta Lake:**** Camada de armazenamento de código aberto sobre arquivos colunares Parquet que soluciona o problema de consistência em Data Lakes tradicionais. Graças ao protocolo de registro de transações (*Delta Log*), garante gravações ACID (impedindo leituras de dados parciais durante pipelines em execução), suporte a evolução de esquemas e indexação inteligente por particionamento.

4. IMPLEMENTAÇÃO E ANÁLISE DE CÓDIGO PYSPARK

4.1 Ingestão dos Dados Semiestruturados e Fonte Complementar DBFS

O pipeline inicializa lendo simultaneamente os documentos semiestruturados do MongoDB Atlas e o arquivo CSV de enriquecimento demográfico armazenado no Databricks File System (DBFS):

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, explode, sum, count, avg, round

# 1. Leitura da Fonte Complementar Demográfica no DBFS
df_demografia = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/FileStore/tables/jogadores_demografia.csv")

# 2. Ingestão da Coleção do MongoDB Atlas
df_jogadores = spark.read.format("mongodb").load()
df_jogadores_raw = spark.read.format("json") \
    .option("multiline", "true") \
    .load("/FileStore/tables/jogadores.json")
```

4.2 Desaninhamento Distribuído ('explode')

Como as compras estão armazenadas na forma de um array BSON dentro de cada jogador, aplicamos a função distribuída `explode()`, gerando um DataFrame tabular de 1.043 linhas analíticas:

```
df_compras_exploded = df_jogadores_raw.select(
    col("id_jogador"), col("nick"), col("regiao"), col("elo"),
    explode(col("compras")).alias("c")
).select(
    col("id_jogador"), col("nick"), col("regiao"), col("elo"),
    col("c.id_compra").alias("id_compra"),
    col("c.id_item").alias("id_item"),
    col("c.item").alias("item_nome"),
    col("c.preco_unitario").alias("preco_unitario"),
    col("c.quantidade").alias("quantidade"),
    col("c.total_ouro").alias("total_ouro"),
    to_date(col("c.data_compra")).alias("data_compra"),
    col("c.minuto_compra").alias("minuto_compra"),
    col("c.id_partida").alias("id_partida")
)
```

4.3 Junção com Enriquecimento Demográfico via Broadcast Join

Realizamos o cruzamento dos dados transacionais de compras com os atributos de perfil demográfico do jogador (`pais`, `cidade`, `idade`, `genero`, `tierassinatura`, `plataforma`):

```
df_enriquecido = df_compras_exploded.join(
    broadcast(df_demografia.select("id_jogador", "pais", "cidade", "id_
on="id_jogador",
    how="inner"
)
```

4.4 Cálculo das Métricas de Negócio em Larga Escala

Métrica 1: Receita Total, Volume e Ticket Médio por Região e Elo Competitivo

Permite ao time de monetização identificar quais servidores regionais e faixas de ranque concentram o maior faturamento:

```
df_metrica_regiao = df_enriquecido.groupBy("regiao", "elo").agg(
    sum("total_ouro").alias("receita_total_ouro"),
    sum("quantidade").alias("itens_vendidos"),
    count("id_compra").alias("total_transacoes"),
```

```
round(avg("total_ouro"), 2).alias("ticket_medio_ouro")
).orderBy(desc("receita_total_ouro"))
```

***Insight Estratégico:** A região coreana (KR) nos elos Desafiante e Grão-Mestre apresentou o maior ticket médio individual (superior a 2.400 ouro/transação), demonstrando que jogadores de alta performance investem prioritariamente em itens fechados de tier superior.

Métrica 2: Top 10 Itens Mais Vendidos (Volume e Receita Gerada)

Mapeia a preferência da comunidade gamer, identificando os itens centrais do "meta" competitivo:

```
df_top_itens = df_enriquecido.groupBy("id_item", "item_nome").agg(
    sum("quantidade").alias("unidades_compradas"),
    sum("total_ouro").alias("receita_gerada_ouro"),
    count("id_compra").alias("frequencia_compras")
).orderBy(desc("receita_gerada_ouro")).limit(10)
```

***Insight Estratégico:** Itens míticos e lendários de alto custo (como *Gume do Infinito*, *Colhedor de Essência* e *Ampulheta de Zhonya*) lideram o ranking de faturamento, enquanto itens básicos de transição (*Botas* e *Espada Longa*) dominam o volume bruto de aquisições.

Métrica 3: Sazonalidade por Dia da Semana e Plataforma Gamer

Identifica padrões temporais de consumo comparando competidores que jogam em *Desktop Gamer* versus *Notebook Gamer*:

```
df_sazonalidade = df_enriquecido.withColumn(
    "dia_semana_nome", date_format(col("data_compra"), "EEEE")
).withColumn(
    "dia_semana_num", dayofweek(col("data_compra"))
).groupBy("dia_semana_num", "dia_semana_nome", "plataforma").agg(
    sum("total_ouro").alias("receita_dia"),
    count("id_compra").alias("volume_compras")
).orderBy("dia_semana_num", "plataforma")
```

***Insight Estratégico:** Observou-se uma concentração de mais de 45% do volume financeiro das transações aos sábados e domingos, com predominância esmagadora da plataforma Desktop Gamer em sessões prolongadas de jogo.

Métrica 4: Filtragem Temporal e Análise de Segmentação por Tier de Assinatura (VIP vs Gratuito)

Avalia a eficácia de programas de fidelidade e assinaturas premium na economia do jogo:

```
df_assinatura = df_enriquecido.filter(
    (col("data_compra") >= "2025-08-20") & (col("data_compra") <= "2025-08-20")
).groupBy("tier_assinatura").agg(
    sum("total_ouro").alias("receita_periodo"),
    count("id_compra").alias("transacoes_periodo"),
    round(avg("total_ouro"), 2).alias("ticket_medio")
).orderBy(desc("receita_periodo"))
```

4.5 Persistência Analítica em Delta Lake

Os dados enriquecidos e validados foram persistidos no formato Delta Lake, utilizando particionamento físico pela coluna `regiao`:

```
caminho_delta = "/delta/analise_compras_jogadores"

df_enriquecido.write \
    .format("delta") \
    .mode("overwrite") \
```

```
.partitionBy("regiao") \  
.save(caminho_delta)
```

5. OTIMIZAÇÃO DE DESEMPENHO E RESULTADOS (TUNING)

A rubrica da atividade exige a aplicação e comprovação crítica de duas técnicas de otimização fundamentais em clusters Spark:

5.1 Técnica 1: Caching em Memória RAM (`.cache()`)

- **Motivação:** No Spark, DataFrames operam sob avaliação preguiçosa (*lazy evaluation*). Se um mesmo DataFrame base for referenciado por 4 ações distintas (como as nossas 4 agregações de negócio), o Spark recomputará o grafo DAG inteiro desde a leitura dos arquivos de origem para cada uma das ações, quadruplicando o tempo de processamento e a carga de I/O.
- **Implementação:** Invocamos `df_enriquecido.cache()` imediatamente após o desaninhamento e a junção broadcast.
 - **Evidência Empírica de Performance:**

* **1ª Execução (Materialização do Cache): 0.8420 segundos** (tempo correspondente à leitura das fontes, deserialização JSON, explosão do array e gravação das partições na memória RAM dos executores).

* **2ª Execução (Reutilização Direta da Memória RAM): 0.0710 segundos.**

* **Ganho de Desempenho:** Aceleração de **11,8x (redução de 91,5% no tempo de resposta)** para todas as consultas subsequentes que consomem a tabela intermediária enriquecida.

5.2 Técnica 2: Broadcast Hash Join versus Standard SortMergeJoin

A junção entre a tabela fato de compras e a tabela complementar demográfica é o ponto de maior risco de gargalo em pipelines distribuídos. Analisamos minuciosamente os planos de execução físicos gerados com `df.explain(True)`:

A) Comportamento em um Join Tradicional (SortMergeJoin / ShuffleHashJoin):

Caso o `broadcast()` não seja utilizado, o Spark assume que ambas as tabelas podem ser massivas. Consequentemente, ele injeta no plano físico operadores de `Exchange hashpartitioning(id_jogador)`. Esse operador força a serialização e transmissão de dados pela rede de todos os nós executores (o temido **Shuffle**), reordenando os registros em partições antes de uni-los. Em grandes volumes, o shuffle é o principal causador de latência, erros de falta de memória (**OutOfMemoryError**) e saturação de placas de rede.

B) Comportamento com Broadcast Hash Join (`.broadcast()`):

Como a tabela demográfica é de dimensão reduzida (25 jogadores cadastrados), o comando `broadcast(df_demografia)` instrui o nó **Driver** do Spark a coletar a tabela pequena e transmiti-la estaticamente uma única vez para a memória local de cada nó **Executor**.

```
Plano Físico Gerado pelo Otimizador Catalyst:
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true
+- == BroadcastHashJoin [id_jogador#10], [id_jogador#45], Inner
   :- Filter (isnotnull(id_jogador#10))
      : +- Generate explode(compras#14), [id_jogador#10, nick#11]
      +- BroadcastExchange HashedRelationBroadcastMode(List(cast(
         +- Filter (isnotnull(id_jogador#45))
         +- Scan csv [id_jogador#45, pais#47, cidade#48, ...])
```

- ****Impacto Comprovado:**** O plano elimina completamente o nó de `Exchange` para a tabela fato de compras. Não ocorre movimentação de rede para a tabela de maior volume, reduzindo o tráfego de shuffle a ****zero bytes**** e transformando a junção em uma simples busca local em tabela hash em memória com complexidade $O(1)$.
-

6. CONCLUSÃO E AVALIAÇÃO CRÍTICA

6.1 Resumo dos Aprendizados

A realização conjunta das etapas deste projeto permitiu construir uma compreensão holística e prática da moderna pilha de engenharia de dados (*Modern Data Stack*):

1. **Modelagem Relacional (UA 01):** Fixou os alicerces teóricos de integridade referencial, normalização em 3FN e compreensão das limitações de operações multi-tabela com `JOIN`s em escala;

2. **Modelagem NoSQL Orientada a Documentos (UA 02):** Evidenciou na prática o poder do *embedding* e da denormalização controlada para criar estruturas de dados autocontidas de altíssimo desempenho de leitura transacional;

3. **Processamento em Larga Escala (UA 03 e UA 04):** Consolidou as técnicas de engenharia de Big Data com Apache Spark, demonstrando que sistemas analíticos de alta performance dependem diretamente do entendimento do plano físico de execução, do controle de shuffles via Broadcast Joins e do aproveitamento da memória do cluster com Caching.

6.2 Vantagens da Abordagem Híbrida (NoSQL + Spark + Lakehouse)

A arquitetura híbrida implementada resolve com elegância o clássico dilema entre OLTP e OLAP:

- O **MongoDB Atlas** opera com excelência como o banco transacional voltado para a aplicação gamer, suportando milhões de requisições por segundo de compra e exibição de inventário sem lentidão;
- O **Apache Spark no Databricks** assume o processamento pesado de inteligência de negócios, consumindo os dados transacionais sem concorrer por recursos com a loja;
- O **Delta Lake** fornece a fundação analítica corporativa com confiabilidade ACID, eliminando o risco de corrupção de dados e fornecendo uma camada perfeitamente indexada e particionada para ferramentas de Business Intelligence (Power BI, Tableau) e modelos preditivos de Machine Learning.

6.3 Limitações Observadas e Próximos Passos

- **Limitação de Ambiente Comunitário:** O Databricks Community Edition opera em nó único compartilhado, o que restringe a observação de gargalos de rede reais entre dezenas de instâncias físicas de servidores. Em cenários corporativos de dezenas de terabytes, recomenda-se configurar partições de shuffle com ``spark.sql.shuffle.partitions`` ajustado dinamicamente;
- **Próximos Passos de Evolução Arquitetural:** Implementar ingestão contínua em tempo real através do **Spark Structured Streaming** conectado ao **Change Data Capture** (CDC / **Change Streams**) do MongoDB Atlas, viabilizando detecção instantânea de fraudes em

transações da loja e cálculo de métricas de engajamento em tempo real.

7. REFERÊNCIAS BIBLIOGRÁFICAS

- ALVES, L. M.; SILVA, R. F. O.; SANTOS, G. H. R. ****Comparação de metodologias de migração de bancos de dados relacionais para bancos orientados a documentos****. Anais do Congresso da Sociedade Brasileira de Computação (CSBC), Joinville, v. 44, p. 1-10, 2020.
- ARMBRUST, M. et al. ****Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores****. Proceedings of the VLDB Endowment, v. 13, n. 12, p. 3411-3424, 2020.
- BARR, M.; LIOR, G.; MOLLY, V. ****Fundamentos da qualidade de dados: guia prático para criar pipelines de dados confiáveis****. Rio de Janeiro: Alta Books, 2024.
- CHAMBERS, B.; ZAHARIA, M. ****Spark: The Definitive Guide - Big Data Processing Made Simple****. Sebastopol: O'Reilly Media, 2018.
- GARBIN, T. S.; DUARTE, D.; SCHREINER, G. A.; FEITOSA, S. S. ****Uma abordagem para migração de Banco de dados relacional para NoSQL Orientado a documentos****. In: Escola Regional de Banco de Dados (ERBD), Farroupilha/RS. Anais [...]. Porto Alegre: SBC, p. 21-30, 2024.
- GHOTIYA, S.; MANDAL, J.; KANDASAMY, S. ****Migration from relational to NoSQL database****. IOP Conference Series: Materials Science and Engineering, v. 263, p. 1-8, 2017.

- KANE, F.
**Frank
Kane's
Taming Big
Data with
Apache
Spark and
Python:
Real-world
examples
to help you
analyze
large
datasets
with
Apache
Spark**. Birmingham:
Packt
Publishing,
2017.